Test Strategy Document based on Domain using chain of thoughts approach

Domain Selected: Account Payable/Receivable
Input prompt provided:
Context-As a Lead functional tester with good Knowledge on BFSI Account Payable/receivable system, Generate me testcase to validate the system
Instruction-Ask me ten questions one by one related to test case generation format Ask me the next question after i have answered the previous one .Test cases should be generated with the following columns
-TestCaseID
-TestCaseName
-TestCaseDescription
-Expected result
-Can be Automated?

Great — let's build this step by step so the test cases match your expectations precisely.
Question 1:
What type of testing are you primarily targeting for this Accounts Payable/Receivable system?
Options (you can choose one or multiple, or specify your own):

- Functional testing
- Integration testing
- Regression testing
- UAT (User Acceptance Testing)
- API testing
- End-to-end business flow testing

Your answer will help shape the scope and depth of the test cases.
Response from My end: Functional
Got it — we'll focus on **Functional Testing** ✅
Question 2:
Which module(s) of the Accounts Payable/Receivable system should the test cases cover?
You can pick one or multiple, for example:

- Vendor Invoice Processing (AP)
- Customer Invoicing (AR)
- Payment Processing (outgoing/incoming)
- Invoice Matching (2-way / 3-way matching)

- Credit/Debit Notes
- Aging & Reporting
- GL Posting / Accounting Entries
- Collections & Dunning

Please list the modules you want included.

Response from My end: Customer Invoicing and Payment processing

Perfect — we'll focus on **Customer Invoicing (AR)** and **Payment Processing** 👍

Question 3:

What level of detail do you want in the test cases?

Choose one:

- High-level (business scenario based, fewer steps)
- Medium-level (clear scenarios with some validation detail)
- Detailed (step-by-step with validations, edge cases, and field-level checks)

This will determine how granular the test cases are.

Response from My end : Detailed (step-by-step with validations, edge cases, and field-level checks)

Got it — we'll go for **detailed** test cases, including step-by-step instructions, validations, and edge cases.

Question 4:

For **Customer Invoicing**, do you want to validate specific invoice generation scenarios like:

- Creating a single invoice
- Creating invoices for multiple items
- Invoice with discounts or tax calculations
- Invoices with different payment terms (e.g., 30 days, 60 days)
- Handling credit notes and adjustments

Or should we focus on a more general validation of the invoicing functionality?

Response from My end :

Invoice with discounts or tax calculations

Invoices with different payment terms

Great, we'll focus on **Invoices with discounts or tax calculations** and **Invoices with different payment terms**. These scenarios will involve verifying the correct application of discounts, taxes, and the calculation of amounts, as well as ensuring the payment terms are correctly applied.

Question 5:

For **Payment Processing**, which types of payments do you need test cases for?

Options (you can specify more if needed):

- Manual payment entry
- Automated payment via ACH/Bank transfer
- Credit card payments

- Payment via electronic funds transfer (EFT)
- Partial payments
- Overpayment / Underpayment
- Refunds or reversals

Which of these are you interested in validating?

Response from My end :

Credit card payments

Partial payments

Got it — we'll include **Credit card payments** and **Partial payments** scenarios for **Payment Processing**.

Question 6:

What system behaviors or business rules need to be validated in the test cases?

For example:

- Correct calculation of tax and discounts based on invoice items
- Handling of payment failures (e.g., declined credit card)
- Correct application of partial payments (remaining balance, status updates)
- Payment reconciliation with open invoices
- Handling of payment terms (due date, grace periods)

Could you list the key behaviors or rules that must be validated for these processes?

Response from My end :

all of the above

Great! We'll include all of those behaviors and business rules in the test cases to ensure comprehensive coverage.

Question 7:

What environments will the system be tested in?

For example:

- Test/QA Environment
- Staging/Pre-prod Environment
- Live/Production Environment (with mock data)
- UAT Environment

This will help us define the scope of testing and ensure proper validation of data and system behavior. Let me know which ones apply.

Response from My end : Test/QA Environment

Perfect, we'll focus on the **Test/QA Environment** for the test cases.

Question 8:

What types of user roles or access levels need to be considered for these test cases?

For example:

- Administrator (full access)
- Accounts Receivable user (limited access to invoicing)
- Payment Processor (access to payment processing only)

- Customer (view-only access to invoices)
- Auditors (read-only access to invoices and payment history)

This will help tailor test cases to different user permissions and scenarios. Let me know which roles we need to include.

Response from My end :

Administrator

Got it — we'll tailor the test cases to the **Administrator** role, which typically has full access to all features.

Question 9:

Do you need test cases to validate any specific integration points with other systems (e.g., ERP, Banking system, CRM, etc.) for Customer Invoicing or Payment Processing? Examples could include:

- Integration with the **ERP system** for general ledger updates
- **Bank API** for payment processing
- **CRM system** for customer data sync
- **Tax calculation service** for applying taxes to invoices

Please let me know if there are any integrations that need to be covered.

Response from My end : not required

Understood — no integrations will be covered in these test cases.

Question 10:

Do you have any specific test data requirements, such as:

- Specific customer or invoice values (e.g., high-value invoices, small invoices)
- Different currencies or exchange rate validations
- Test cases involving edge cases like invalid data input (e.g., invalid credit card number, expired card, etc.)
- Bulk invoice creation or bulk payment scenarios
- Specific dates (e.g., end of month, leap years)

Let me know if any of these data scenarios need to be included in the test cases.

Response from My end :

all cases except edge cases/invalid data

Got it — we'll include **high-value invoices**, **small invoices**, **different currencies/exchange rate validations**, **bulk invoice creation**, and **specific dates** (like end of month or leap years), but we'll exclude edge cases/invalid data.

Now that I have all the information, I'll generate the test cases. I'll follow the format you mentioned:

- **TestCaseID**
- **TestCaseName**
- **TestCaseDescription**
- **Expected result**

- **Can be Automated?**

Let me know if you'd like to start with a specific module or if you want me to cover both **Customer Invoicing** and **Payment Processing** in the first batch of test cases.
Response from My end :
we can go with customer invoicing first
Great! Let's start with **Customer Invoicing**.
Here's a set of test cases based on the information you provided. Each one will follow the format you requested:

Test Case 1
TestCaseID: TC_AR_01
TestCaseName: Create Invoice with Discount and Tax Calculation
TestCaseDescription:
Verify that a customer invoice is correctly generated with a discount applied and taxes calculated on the items.

- The invoice should reflect the correct discounted price and tax values.
- The system should apply the discount and tax rules based on the invoice items.

Expected result:

- Invoice created with correct discount (e.g., 10% off)
- Tax calculated based on applicable rates
- Total amount reflects both discount and tax correctly

Can be Automated? Yes

Test Case 2
TestCaseID: TC_AR_02
TestCaseName: Create Invoice with Multiple Items and Different Taxes
TestCaseDescription:
Verify that an invoice with multiple items, each having different tax rates, is generated correctly.

- Ensure the correct tax rate is applied to each item.
- The total tax should be the sum of individual item taxes.

Expected result:

- Each item displays the correct tax based on its tax rate
- The total tax amount at the bottom of the invoice is correct
- The final invoice amount is accurate (item total + tax)

Can be Automated? Yes

Test Case 3
TestCaseID: TC_AR_03
TestCaseName: Invoice Generation with Different Payment Terms

TestCaseDescription:
Verify that an invoice is generated with different payment terms (e.g., Net 30, Net 60) and that the due date is calculated correctly.

- The system should correctly calculate the due date based on the invoice creation date and payment terms.

Expected result:

- The correct due date is displayed on the invoice based on the payment term (e.g., 30 days after invoice creation)
- Invoice status shows “Pending” until payment is made

Can be Automated? Yes

Test Case 4
TestCaseID: TC_AR_04
TestCaseName: Invoice with Payment Term Adjustment
TestCaseDescription:
Verify that the system correctly handles an adjustment to the payment term (e.g., extending from Net 30 to Net 60) after the invoice is created.

- The system should adjust the due date and reflect this change on the invoice.

Expected result:

- The due date updates based on the new payment term
- The invoice status and terms reflect the change correctly

Can be Automated? Yes

Test Case 5
TestCaseID: TC_AR_05
TestCaseName: Create Invoice for High-Value Items
TestCaseDescription:
Verify that the system can handle high-value invoices (e.g., invoices over $100,000).

- The invoice should be generated correctly with proper calculation for tax, discounts, and payment terms.

Expected result:

- Invoice amount should be correctly calculated even for high-value transactions
- Tax and discount values should be correctly applied
- The system should not crash or throw errors due to the high invoice amount

Can be Automated? Yes

Test Case 6
TestCaseID: TC_AR_06
TestCaseName: Create Invoice for Small-Value Items

TestCaseDescription:
Verify that the system correctly handles low-value invoices (e.g., below $10).

- The invoice should still calculate tax and discounts correctly.

Expected result:

- Invoice should be generated with proper tax and discount, even for small amounts
- No rounding errors or issues with low-value transactions

Can be Automated? Yes

Test Case 7
TestCaseID: TC_AR_07
TestCaseName: Generate Invoice with Currency Conversion (Multi-Currency)
TestCaseDescription:
Verify that the system can handle invoices in different currencies and apply correct exchange rates for tax and totals.

- The system should convert the invoice amount and tax according to the current exchange rate.

Expected result:

- The invoice displays the correct amount in the specified currency
- Exchange rate is correctly applied to both the invoice total and tax calculations

Can be Automated? Yes

Test Case 8
TestCaseID: TC_AR_08
TestCaseName: Generate Invoice on Leap Year Date
TestCaseDescription:
Verify that invoices created on a leap year (e.g., February 29th) are processed correctly.

- The invoice should correctly calculate the due date based on payment terms, even in a leap year.

Expected result:

- The due date should be correctly set even if the invoice is created on February 29th
- The system should not miscalculate the due date due to the leap year

Can be Automated? Yes

Test Case 9
TestCaseID: TC_AR_09
TestCaseName: Create Bulk Invoice (Multiple Customers)
TestCaseDescription:
Verify that the system can generate multiple invoices in bulk for different customers.

- Each invoice should have its own correct customer details, items, taxes, and payment terms.

Expected result:

- Multiple invoices are created with accurate details for each customer
- Each invoice has the correct payment terms, tax, and discount values

Can be Automated? Yes

Test Case 10
TestCaseID: TC_AR_10
TestCaseName: Verify Invoice with Credit Note Adjustment
TestCaseDescription:
Verify that an invoice can be adjusted with a credit note.

- The credit note should be properly applied, reducing the invoice balance.

Expected result:

- Credit note correctly applied to the invoice
- The balance on the invoice is updated with the credit amount deducted

Can be Automated? Yes

This covers a range of scenarios for **Customer Invoicing** based on your requirements. Would you like to proceed with generating test cases for **Payment Processing** next, or would you like any changes or additions to the current test cases?
Response from My end :
generate the outcome in tabular format. As a AX domain expert review the generated test cases and Add review comments to a new column named review comments

Perfect! I've reviewed the previously generated **Customer Invoicing** test cases from an **MS Dynamics AX / AX domain expert** perspective and compiled them into a **tabular format**. I've also added a **Review Comments** column with domain-specific observations or improvement suggestions.

TestCaseID	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
TC_AR_01	Create Invoice with Discount and Tax Calculation	Verify that a customer invoice is correctly generated with a discount applied and taxes calculated on the items.	Invoice created with correct discount and tax values;	Yes	Ensure AX tax codes and discount structures are correctly mapped; validate on

TestCaseID	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
			total amount reflects both correctly		different customer groups
TC_AR_02	Create Invoice with Multiple Items and Different Taxes	Verify that an invoice with multiple items, each having different tax rates, is generated correctly	Each item displays correct tax; total tax and invoice amount accurate	Yes	Confirm that AX item sales tax groups and VAT setup are validated; edge case for mixed-tax invoices
TC_AR_03	Invoice Generation with Different Payment Terms	Verify invoice is generated with different payment terms (e.g., Net 30, Net 60)	Correct due date displayed; invoice status “Pending”	Yes	Review AX payment term configurations and ensure proper posting to ledger; check for holidays affecting due dates
TC_AR_04	Invoice with Payment Term Adjustment	Verify the system handles adjustment to payment terms after invoice creation	Due date updates; invoice status reflects change	Yes	Validate workflow for updating terms; ensure posting to customer aging and

TestCaseID	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
					collections is accurate
TC_AR_05	Create Invoice for High-Value Items	Verify system handles high-value invoices correctly	Invoice amount calculated correctly; system handles high-value without errors	Yes	Check AX limits on invoice totals, rounding, and posting to ledger; validate workflow approvals for high-value invoices
TC_AR_06	Create Invoice for Small-Value Items	Verify system handles low-value invoices correctly	Invoice generated correctly; tax and discounts applied; no rounding errors	Yes	Ensure AX rounding policies and minimum invoice value configurations are applied
TC_AR_07	Generate Invoice with Currency Conversion (Multi-Currency)	Verify invoices in different currencies and correct exchange rate application	Invoice displays correct amount in specified currency; exchange rate applied correctly	Yes	Validate AX currency setup, exchange rate types (spot vs. daily), and accounting currency conversion
TC_AR_08	Generate Invoice on	Verify invoices created on leap year	Due date correctly	Yes	Review AX calendar

TestCaseID	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
	Leap Year Date	date are processed correctly	set; system calculates due date accurately		setup and date handling; test with different payment terms across leap years
TC_AR_09	Create Bulk Invoice (Multiple Customers)	Verify system can generate multiple invoices in bulk for different customers	Multiple invoices created accurately; each invoice with correct details	Yes	Confirm AX batch job configuration, concurrency limits, and workflow approval routing
TC_AR_10	Verify Invoice with Credit Note Adjustment	Verify invoice can be adjusted with a credit note	Credit note applied; invoice balance updated correctly	Yes	Validate that AX credit note posting updates AR ledger and customer balance; test with partial credit notes

---

✅ **AX Domain Review Highlights:**

1. **Tax & Discount Validation:** Ensure AX tax codes, VAT, and discount policies are correctly configured and applied to each scenario.
2. **Payment Terms & Due Dates:** Include holiday calendars, grace periods, and batch update checks.
3. **High/Low Invoice Values:** Validate system handles extreme amounts with correct rounding and approvals.
4. **Currency Handling:** Check posting to ledger in both transactional and accounting currency.

5. **Batch/Bulk Processing:** Validate workflow approval routing and concurrency handling.
6. **Credit Notes:** Ensure AR ledger, customer balances, and reporting are correctly updated.

I can next generate **Payment Processing** test cases in the same **tabular format with AX domain review comments** so you have a complete set.
Do you want me to do that now?
Response from My end :
Yes
Perfect! Let's generate **Payment Processing** test cases for **Credit Card Payments** and **Partial Payments**, in **tabular format** with **AX domain review comments**.

TestCaseID	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
TC_PP_01	Process Credit Card Payment - Full Invoice	Verify that a customer invoice can be fully paid via credit card	Invoice status updated to "Paid"; AR ledger reflects full payment	Yes	Validate AX payment connector setup for credit card gateway; ensure proper posting to AR and GL accounts
TC_PP_02	Process Credit Card Payment - Partial Payment	Verify that a partial payment via credit card is applied correctly	Invoice balance updated; AR ledger shows partial payment; remaining amount due	Yes	Check that AX allows partial payments and updates customer balance correctly; verify aging reports
TC_PP_03	Credit Card Payment with Tax &	Verify credit card payment on invoice	Payment applied correctly;	Yes	Validate that AX calculates

TestCaseID	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
	Discount Invoice	with discounts and taxes	invoice fully/partially settled based on amount paid		payment allocation correctly between tax, discount, and principal
TC_PP_04	Partial Payment Across Multiple Invoices	Verify a single partial payment applied to multiple invoices	Payment applied proportionally; AR ledger updated; invoice balances correct	Yes	AX allows allocation rules; ensure automatic application rules are consistent with business policy
TC_PP_05	Credit Card Payment with Multi-Currency Invoice	Verify payment applied correctly when invoice currency differs from payment currency	Payment applied; currency conversion correct; AR ledger reflects proper amounts	Yes	Check currency conversion setup in AX; ensure exchange rates applied match posting currency
TC_PP_06	Refund/Credit for Overpayment via Credit Card	Verify system handles overpayment and issues refund correctly	Refund processed; AR ledger and customer balance updated; invoice	Yes	Ensure AX handles overpayments correctly; verify refund posting workflow

TestCaseID	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
			marked as “Closed”		
TC_PP_07	Partial Payment with Payment Term Adjustment	Verify system correctly handles partial payment after payment term change	Partial payment applied; due date adjusted; remaining balance correct	Yes	Validate that payment term adjustments do not affect allocation logic; check AR aging impact
TC_PP_08	Bulk Credit Card Payments	Verify processing multiple credit card payments in batch	Payments applied to respective invoices; AR ledger updated correctly	Yes	Confirm AX batch processing configuration; concurrency limits and posting sequence validated
TC_PP_09	Payment Reversal for Credit Card Transaction	Verify reversal of a credit card payment	Payment reversed; invoice balance restored; AR ledger updated	Yes	AX reversal posting should update AR and GL; check integration with payment gateway for reversals
TC_PP_10	Partial Payment	Verify reports reflect partial payments accurately	Invoice balances correct; AR	Yes	Ensure AX AR reports, customer

TestCaseID	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
	Reporting & Reconciliation		aging reports reflect partial payments; GL reconciliation correct		statements, and GL reconciliation correctly reflect partial payments

---

AX Domain Review Highlights for Payment Processing:

1. **Payment Gateway Integration:** Ensure AX connectors for credit cards are properly configured and test transaction posting.
2. **Partial Payment Handling:** Confirm allocation logic (principal, tax, discount) works correctly across invoices.
3. **Multi-Currency Payments:** Validate currency conversion rates, posting currency, and reconciliation.
4. **Batch & Bulk Processing:** Ensure AX batch jobs handle concurrency, exceptions, and approval workflows.
5. **Reversals & Refunds:** Verify AR ledger, customer balances, and reporting are updated correctly.
6. **Reporting & Reconciliation:** Aging reports, customer statements, and GL reconciliation must reflect partial or full payments accurately.

If you want, I can **combine both Customer Invoicing and Payment Processing into a single master test case matrix** ready for execution in AX with all review comments included. This will give you a complete, AX domain-reviewed test suite in one table.
Do you want me to do that next?
Response from My end :
Yes
Perfect! Here's a **master test case matrix** combining **Customer Invoicing** and **Payment Processing** for AX, in a single table with **detailed test case descriptions, expected results, automation feasibility, and AX domain expert review comments**.

TestCaseID	Module	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
TC_AR_01	Customer Invoicing	Create Invoice with Discount and Tax Calculation	Verify a customer invoice is correctly generated with a discount applied and taxes calculated on items.	Invoice shows correct discount and tax; total amount correct	Yes	Ensure AX tax codes, discount structures, and customer group rules are correctly applied
TC_AR_02	Customer Invoicing	Create Invoice with Multiple Items and Different Taxes	Verify invoice with multiple items, each having different tax rates, is generated correctly	Each item displays correct tax; total tax and invoice amount accurate	Yes	Confirm AX item sales tax groups, VAT setup, and posting rules
TC_AR_03	Customer Invoicing	Invoice Generation with Different Payment Terms	Verify invoice generated with different payment terms (Net 30, Net 60)	Correct due date; invoice status “Pending”	Yes	Review AX payment term setup, holiday calendars, and ledger posting
TC_AR_04	Customer Invoicing	Invoice with Payment Term Adjustment	Verify system handles adjustment to payment terms after invoice creation	Due date updated; invoice status reflects new term	Yes	Validate workflow for payment term changes and AR aging update

TestCaseID	Module	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
TC_AR_05	Customer Invoicing	Create Invoice for High-Value Items	Verify system handles invoices over $100,000	Invoice amount correct; no system errors	Yes	Check AX rounding, posting, approvals, and workflow limits for high-value invoices
TC_AR_06	Customer Invoicing	Create Invoice for Small-Value Items	Verify low-value invoices are generated correctly	Invoice shows correct amount; tax and discount applied	Yes	Validate AX rounding policies and minimum invoice value settings
TC_AR_07	Customer Invoicing	Generate Invoice with Currency Conversion	Verify multi-currency invoices are calculated correctly	Correct invoice amount in specified currency; exchange rate applied	Yes	Confirm currency setup, conversion type, and ledger postings in AX
TC_AR_08	Customer Invoicing	Generate Invoice on Leap Year Date	Verify invoices created on leap year date are processed correctly	Due date correct; invoice processed accurately	Yes	Validate AX calendar handling, leap year date, and payment term calculation

TestCaseID	Module	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
TC_AR_09	Customer Invoicing	Create Bulk Invoice (Multiple Customers)	Verify system generates multiple invoices in bulk for different customers	Multiple invoices created accurately; details correct	Yes	Confirm batch job configuration, concurrency, and workflow approvals in AX
TC_AR_10	Customer Invoicing	Verify Invoice with Credit Note Adjustment	Verify invoice can be adjusted using a credit note	Credit note applied; balance updated	Yes	Validate AR ledger update, partial credit note handling, and reporting
TC_PP_01	Payment Processing	Process Credit Card Payment - Full Invoice	Verify invoice fully paid via credit card	Invoice status “Paid”; AR ledger updated	Yes	Confirm AX payment connector setup, credit card gateway integration, and AR/GL posting
TC_PP_02	Payment Processing	Process Credit Card Payment - Partial Payment	Verify partial payment via credit card	Invoice balance updated; AR ledger shows partial payment	Yes	Check AX allocation logic; aging reports updated correctly

TestCaseID	Module	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
TC_PP_03	Payment Processing	Credit Card Payment with Tax & Discount Invoice	Verify payment on invoice with discounts and taxes	Payment applied correctly; invoice fully/partially settled	Yes	Validate allocation between principal, tax, and discount in AX
TC_PP_04	Payment Processing	Partial Payment Across Multiple Invoices	Verify single partial payment applied to multiple invoices	Payment allocated; AR ledger updated; balances correct	Yes	Confirm AX allocation rules and automatic application consistency
TC_PP_05	Payment Processing	Credit Card Payment with Multi-Currency Invoice	Verify payment applied when invoice currency differs from payment currency	Payment applied; conversion correct; AR ledger reflects amounts	Yes	Validate currency conversion setup, posting currency, and ledger impact
TC_PP_06	Payment Processing	Refund/Credit for Overpayment via Credit Card	Verify overpayment handled and refund issued	Refund processed; AR ledger and customer balance updated	Yes	Check AX overpayment handling and refund posting workflow
TC_PP_07	Payment Processing	Partial Payment with Payment	Verify system handles partial payment after payment term change	Partial payment applied; due date adjusted;	Yes	Ensure payment term change does not

TestCaseID	Module	TestCaseName	TestCaseDescription	Expected Result	Can be Automated?	Review Comments
		Term Adjustment		balance correct		break allocation logic; check AR aging
TC_PP_08	Payment Processing	Bulk Credit Card Payments	Verify multiple credit card payments processed in batch	Payments applied; AR ledger updated	Yes	Validate batch job configuration, concurrency, and approval workflows in AX
TC_PP_09	Payment Processing	Payment Reversal for Credit Card Transaction	Verify reversal of a credit card payment	Payment reversed; invoice balance restored; AR ledger updated	Yes	Ensure AR/GL ledger updated; test gateway reversal integration
TC_PP_10	Payment Processing	Partial Payment Reporting & Reconciliation	Verify reports reflect partial payments accurately	Invoice balances correct; aging reports and GL reconciliation accurate	Yes	Confirm AR reports, customer statements, and GL reconciliation reflect partial payments correctly

---

✅ **Master Review Highlights (AX Domain Expert):**

1. **Tax & Discount Handling:** Validate AX tax codes, VAT, and discount rules across invoices and payments.
2. **Payment Terms & Due Dates:** Include holiday calendars, leap year handling, and term adjustments.
3. **High/Low Value Transactions:** Verify rounding, approvals, and ledger postings.
4. **Multi-Currency Scenarios:** Confirm currency conversion, posting currency, and AR ledger accuracy.
5. **Credit Card Payment Handling:** Full, partial, overpayments, refunds, reversals — integration with payment gateway and ledger posting must be validated.
6. **Batch/Bulk Processing:** Ensure AX handles concurrency, batch job limits, and workflow approvals.
7. **Reporting & Reconciliation:** AR reports, aging, GL reconciliation must reflect payments, partial payments, and adjustments correctly.

If you want, I can **also generate this as a ready-to-use Excel/CSV format** with all columns for direct use in **AX test execution tracking**.
Do you want me to do that next?

Automation framework using chain of thought

Input prompt provided:
Context- As a automation engineer generate a dotnet framework structure to test APIs
Instruction
Ask me five questions one by one related to framework setup
Ask me the next question after i have answered the previous one
[STRICT] Framework should be built with latest stable packages and components
[MANDATORY]Framework must be easily understandable by beginers
[MANDATORY]Framework should hanle dynamic parameter passing for components like headres,baseURL,endpoints
Got it! Let's get started.

First question:
Do you have a preferred test automation tool or library to be used for API testing (such as RestSharp, HttpClient, or a specific framework like xUnit or NUnit), or should I suggest one that aligns with your requirements?
Response Provided By Me: RestSharp
Great choice! RestSharp is a powerful tool for working with APIs.

Second question:
For the framework, do you prefer a certain testing framework for structuring and running tests, like xUnit, NUnit, or MSTest? If you don't have a preference, I can suggest one that fits well with RestSharp.

Response Provided By Me: Nunit
Perfect! NUnit will work seamlessly with RestSharp.
Third question:
Would you like the framework to support different environments (e.g., dev, staging, production) with the ability to easily switch between them, or should it be focused on a single environment for simplicity?
Response Provided By Me: switch between environment
Got it! We'll make sure the framework allows switching between different environments easily.

Fourth question:
How would you like to manage dynamic parameters like headers, base URLs, and endpoints? Would you prefer to store them in a configuration file (e.g., JSON, YAML, or XML) or manage them within the code itself for simplicity?
Response Provided By Me: XML
Great choice! XML is a solid option for configuration and can be easily parsed and managed.

Fifth and final question:
For reporting, would you prefer the framework to generate simple, readable test reports (e.g., in HTML or XML format) or do you have a specific reporting tool in mind (like Allure, ExtentReports, etc.)?
Response Provided By Me: ExtentReports
Awesome! ExtentReports will give you detailed, interactive test reports.

Now that I have all the information, I'll outline a basic framework structure for API testing with RestSharp, NUnit, dynamic parameter handling via XML, environment switching, and ExtentReports integration. Here's how it would be organized:

Folder Structure

```
ProjectName
│
├── Config
│   └── TestConfig.xml          # XML file for environment-specific settings
│
├── Drivers                     # If needed for browser automation or specific drivers
│
├── Pages                       # If your API interactions grow more complex, you can use
Page Object Model
│
├── Reports                     # Output folder for ExtentReports
│   └── Report.html             # Final report after tests
│
├── TestCases                   # NUnit test case classes
│   └── ApiTests.cs
│
├── Utilities                   # Helper classes and utilities
│   ├── ConfigHelper.cs         # For reading and parsing the XML config
│   ├── RestSharpHelper.cs      # For API request/response handling
│   └── ExtentReportHelper.cs   # For setting up and managing ExtentReports
│
└── TestRunner                  # Entry point to run tests
    └── TestRunner.cs
```

Sample Implementation Details:

1. TestConfig.xml

Store dynamic parameters like base URL, headers, and endpoints in an XML file.

```
<config>
  <environments>
    <dev>
      <baseUrl>https://dev.example.com/api</baseUrl>
      <headers>
        <header key="Authorization" value="Bearer abc123" />
        <header key="Content-Type" value="application/json" />
      </headers>
      <endpoints>
        <endpoint name="GetUser" value="/user/{id}" />
        <endpoint name="CreateUser" value="/user" />
```

```
        </endpoints>
    </dev>
    <staging>
        <!-- Similar structure for staging -->
    </staging>
  </environments>
</config>
```

2. ConfigHelper.cs

Handle the reading and parsing of the XML config.

```
using System;
using System.Xml.Linq;

public class ConfigHelper
{
    private static XElement _config;

    static ConfigHelper()
    {
        _config = XElement.Load("Config/TestConfig.xml");
    }

    public static string GetBaseUrl(string environment)
    {
        return
_config.Element("environments").Element(environment).Element("baseUrl").Value;
    }

    public static string GetHeaderValue(string environment, string key)
    {
        var header = _config.Element("environments").Element(environment)
            .Element("headers").Element("header[@key='" + key + "']");
        return header?.Attribute("value")?.Value;
    }

    public static string GetEndpoint(string environment, string endpointName)
    {
        return _config.Element("environments").Element(environment)
            .Element("endpoints").Element("endpoint[@name='" + endpointName +
"']").Value;
    }
```

```
}
```

3. RestSharpHelper.cs

Simplify API calls using RestSharp.

```
using RestSharp;
using System;

public class RestSharpHelper
{
    private RestClient _client;

    public RestSharpHelper(string baseUrl)
    {
        _client = new RestClient(baseUrl);
    }

    public IRestResponse ExecuteRequest(string endpoint, Method method, object body
= null, Dictionary<string, string> headers = null)
    {
        var request = new RestRequest(endpoint, method);

        if (headers != null)
        {
            foreach (var header in headers)
            {
                request.AddHeader(header.Key, header.Value);
            }
        }

        if (body != null)
        {
            request.AddJsonBody(body);
        }

        return _client.Execute(request);
    }
}
```

4. TestRunner.cs

Setup the test runner for NUnit to configure environment and ExtentReports.

```
using NUnit.Framework;
using ExtentReports;
```

```
[TestFixture]
public class TestRunner
{
    private static ExtentReports _extentReports;

    [OneTimeSetUp]
    public void SetupReports()
    {
        _extentReports = new ExtentReports();
        _extentReports.AttachReporter(new ExtentHtmlReporter("Reports/Report.html"));
    }

    [OneTimeTearDown]
    public void TearDownReports()
    {
        _extentReports.Flush();
    }

    public static ExtentTest CreateTest(string testName)
    {
        return _extentReports.CreateTest(testName);
    }
}
```

5. ApiTests.cs

The main test cases for API testing using RestSharp and ExtentReports.

```
using NUnit.Framework;
using RestSharp;
using ExtentReports;

[TestFixture]
public class ApiTests : TestRunner
{
    private RestSharpHelper _restSharpHelper;
    private string _baseUrl;

    [SetUp]
    public void Setup()
    {
        // Set up environment (could be dev, staging, etc.)
```

```
        _baseUrl = ConfigHelper.GetBaseUrl("dev");
        _restSharpHelper = new RestSharpHelper(_baseUrl);
    }

    [Test]
    public void TestGetUser()
    {
        var endpoint = ConfigHelper.GetEndpoint("dev", "GetUser");
        var response = _restSharpHelper.ExecuteRequest(endpoint, Method.GET);

        var test = CreateTest("TestGetUser");

        Assert.AreEqual(200, (int)response.StatusCode);
        test.Pass("GET /user request passed with status " + response.StatusCode);
    }

    [Test]
    public void TestCreateUser()
    {
        var endpoint = ConfigHelper.GetEndpoint("dev", "CreateUser");
        var headers = new Dictionary<string, string>
        {
            { "Authorization", ConfigHelper.GetHeaderValue("dev", "Authorization") }
        };

        var user = new { name = "John", age = 30 };
        var response = _restSharpHelper.ExecuteRequest(endpoint, Method.POST, user,
headers);

        var test = CreateTest("TestCreateUser");

        Assert.AreEqual(201, (int)response.StatusCode);
        test.Pass("POST /user request passed with status " + response.StatusCode);
    }
}
```

Next Steps

1. Create the TestConfig.xml with your environment settings (base URL, headers, and endpoints).
2. Set up the RestSharp, NUnit, and ExtentReports dependencies via NuGet.

3. Write tests for all your API endpoints using the structure above.

Would you like help with setting up the project or any specific part of this?